

Trường ĐH Công Thương TP. HCM Khoa Công nghệ Thông tin Bộ môn Kỹ thuật phần mềm CÔNG NGHỆ .NET	BÀI 10. LẬP TRÌNH VỚI CƠ SỞ DỮ LIỆU (tiếp theo)	
--	--	--

A. MỤC TIÊU

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Hiểu và vận dụng được các trạng thái hoạt động của EF.
- Thực hiện các thao tác thêm – xóa – sửa (CRUD) trên cơ sở dữ liệu bằng LINQ to Entities theo đúng quy trình của EF.
- Xây dựng giao diện quản lý dữ liệu hoàn chỉnh, kết hợp DataGridView, TextBox, ComboBox và các nút chức năng (Thêm, Sửa, Xóa, Lưu).

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SINH VIÊN

STT	Thiết bị - Phần mềm – Thư viện	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	
2	Microsoft Visual Studio 2019 trở lên (.NET Framework từ 4.7.2)	1	1	
3	Entity Framework (6.x.x)	1	1	
4	Microsoft SQL Server từ 2017	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

Chương 4. Lập trình với cơ sở dữ liệu (tiếp theo)

4.3. Thao tác CRUD với EF

a. Thêm – xóa – sửa dữ liệu

Các thao tác này sử dụng các phương thức có sẵn trên DbSet<T> và được xác nhận bằng SaveChanges().

▪ Thêm dữ liệu:

Các bước thực hiện:

- Khởi tạo đối tượng Entity mới.
- Thêm đối tượng vào tập hợp (DbSet) bằng phương thức db.DbSet.Add(entity).

- Khi Add được gọi, trạng thái của Entity chuyển thành Added.
- Gọi db.SaveChanges(). EF tạo và thực thi lệnh INSERT.

Ví dụ:

```
var k = new Khoa() { MaKhoa = "CNTT",
                    TenKhoa = "Công nghệ thông tin" };
db.Khoas.Add(k);
db.SaveChanges();
LoadData();
```

▪ Sửa dữ liệu:

Các bước thực hiện (Sửa Entity đang được theo dõi)

- Truy vấn Entity từ DbContext (db.DbSet.Find(id)).
- Thay đổi giá trị của các thuộc tính.
- Gọi db.SaveChanges(). EF tự động nhận diện thay đổi, chuyển trạng thái thành Modified, và thực thi lệnh UPDATE.

Ví dụ:

```
var k = db.Khoas.Find("CNTT");
k.TenKhoa = "Công nghệ thông tin - cập nhật";
db.SaveChanges();
LoadData();
```

▪ Xóa dữ liệu:

Các bước thực hiện (Xóa Entity đang được theo dõi)

- Tìm Entity cần xóa (bằng Find hoặc truy vấn LINQ).
- Sử dụng phương thức db.DbSet.Remove(entity).
 - Trạng thái của Entity chuyển thành Deleted.
- Gọi db.SaveChanges(). EF tạo và thực thi lệnh DELETE.

Ví dụ:

```
var k = db.Khoas.Find("CNTT");
db.Khoas.Remove(k);
db.SaveChanges();
LoadData();
```

Các trạng thái của Entity trong Entity Framework:

Trạng thái	Ý nghĩa
Added	Đối tượng mới, sẽ được INSERT vào DB
Modified	Đối tượng đã thay đổi, sẽ UPDATE
Deleted	Đối tượng bị xóa, sẽ DELETE
Unchanged	Không thay đổi
Detached	Không được DbContext theo dõi

b. Sử dụng raw SQL queries

Mặc dù EF được thiết kế để tránh SQL thuần, nhưng đôi khi cần thiết phải sử dụng SQL thuần trong các trường hợp sau:

- Tối ưu hiệu suất: Khi truy vấn quá phức tạp, việc viết SQL thuần có thể nhanh hơn.
- Thao tác với Stored Procedure: Gọi các Stored Procedure có sẵn trong CSDL.
- Thao tác quản trị CSDL: Các lệnh không phải CRUD (ví dụ: CREATE TABLE, ALTER).

EF 6 hỗ trợ hai phương thức chính để làm việc với raw SQL:

 **Truy vấn dữ liệu (SELECT) bằng SQL – Database.SqlQuery<T>()**

Cú pháp:

```
var result = db.Database.SqlQuery<T>("SQL query", params).ToList();
```

- T: kiểu dữ liệu trả về (Entity hoặc DTO).

 **Lưu ý khi sử dụng Raw SQL trong EF6:**

- Chỉ sử dụng khi thật sự cần thiết.
- Luôn truyền tham số (@p0, @p1) để tránh SQL Injection.
- Không nối chuỗi SQL trực tiếp từ input người dùng.
- Kết quả SELECT phải khớp với entity/model.
- Ưu tiên LINQ vì dễ bảo trì và an toàn hơn.

Ví dụ 1: Lấy danh sách Khoa

```
var list = db.Database.SqlQuery<Khoa>("SELECT * FROM Khoa").ToList();
```

Ví dụ 2: Lọc sinh viên theo khoa (có tham số)

```
int makhoa = 1;
var sv = db.Database.SqlQuery<SinhVien>(
    "SELECT * FROM SinhVien WHERE MaKhoa = @p0", makhoa
).ToList();
```

 **Thực thi câu lệnh không trả về dữ liệu (INSERT, UPDATE, DELETE)**

Dùng **Database.ExecuteNonQuery()**.

Cú pháp:

```
int affectedRows = db.Database.ExecuteNonQuery("SQL
command", params);
```

Ví dụ: Cập nhật điểm sinh viên

```
db.Database.ExecuteNonQuery(
    "UPDATE KetQua SET Diem = @p0
    WHERE MaSinhVien = @p1 AND MaMonHoc = @p2",
    diem, maSV, maMon
);
```

 **Gọi Stored Procedure**

Nếu database đã có stored procedure:

```
CREATE PROCEDURE GetSinhVienByKhoa @MaKhoa INT
AS
BEGIN
    SELECT * FROM SinhVien WHERE MaKhoa = @MaKhoa
END
```

EF gọi như sau:

```
var result = db.Database.SqlQuery<SinhVien>(
    "EXEC GetSinhVienByKhoa @p0", maKhoa
).ToList();
```

Lưu ý khi sử dụng Raw SQL trong EF6:

- Chỉ dùng raw SQL khi cần; ưu tiên LINQ vì dễ bảo trì và an toàn.
- Khi SELECT phải khớp với entity hoặc model (tên cột tương ứng).
- Nên truyền tham số để chống SQL Injection.
- Không dùng raw SQL cho các thao tác đơn giản (thêm, xóa, sửa) vì LINQ tiện hơn.

2. Bài tập thực hành trên lớp

Bài tập có hướng dẫn

Bài 1. Ứng dụng quản lý Khoa có giao diện như sau (sử dụng bài tập hướng dẫn 1 trong bài thực hành 09)

Mã khoa	Tên khoa
CNTP	Công nghệ thực phẩm
CNTT	Công nghệ thông tin
CT	Chính trị
HH	Hóa học
KT	Kế toán

Yêu cầu:

- Thực hiện chức năng cập nhật dữ liệu (Thêm/ Xóa/ Sửa) bảng Khoa
- Dữ liệu phải được đồng bộ tự động:
 - Khi chọn dòng trên DataGrid thì các TextBox tương ứng sẽ hiển thị đúng dữ liệu.
 - Khi chọn Button Thêm/Sửa/Xóa, DataGrid cập nhật lại nội dung và đồng thời dữ liệu được cập nhật xuống CSDL.

Hướng dẫn thực hiện:

Bước 1: Các yêu cầu cần chuẩn bị trước:

- Giao diện QLKhoaView.xaml
- DbContext: Lớp QLSinhVienEntities (được sinh ra từ EF6).
- Lớp `KhoaViewModel`: `BaseViewModel`

Bước 2: Tạo class `RelayCommand.cs` thực thi interface `ICommand`

Bước 3: Mở lớp `KhoaViewModel.cs` (đã được tạo trong project trước) viết thêm một số phương thức để thực hiện các chức năng cập nhật dữ liệu:

Bổ sung:

- Thuộc tính NewKhoa
`public Khoa NewKhoa { get; set; } = new Khoa();`

- Các properties Command

```
public RelayCommand AddCommand { get; set; }  
public RelayCommand DeleteCommand { get; set; }  
public RelayCommand UpdateCommand { get; set; }
```

- Property SelectedKhoa

```
public Khoa SelectedKhoa {  
    get => _SelectedKhoa;  
    set {  
        _SelectedKhoa = value;  
        OnPropertyChanged(nameof(SelectedKhoa));  
        if (SelectedKhoa != null) {
```

```

        NewKhoa.MaKhoa = SelectedKhoa.MaKhoa;
        NewKhoa.TenKhoa = SelectedKhoa.TenKhoa;
        // Báo UI cập nhật TextBox
        OnPropertyChanged(nameof(NewKhoa));
    }
}

```

- Trong phương thức `KhoaViewModel()`

```

// ===== KHỞI TẠO COMMAND =====
AddCommand = new RelayCommand(o => Add());
DeleteCommand = new RelayCommand(o => Delete(),
                                o => SelectedKhoa != null);
UpdateCommand = new RelayCommand(o => Update(),
                                o => SelectedKhoa != null);

```

- Phương thức `LoadData()`

```

void LoadData()
{
    DS_Khoa = new ObservableCollection<Khoa>(db.Khoas.ToList());
    OnPropertyChanged(nameof(DS_Khoa));
    SelectedKhoa = DS_Khoa.FirstOrDefault();
}

```

- Các phương thức `Add()`, `Delete()`, `Update()`

```

private void Add() {
    if (NewKhoa.MaKhoa == "" || NewKhoa.TenKhoa == "") {
        MessageBox.Show(
            "Dữ liệu không hợp lệ.\nVui lòng kiểm tra lại các trường.",
            "Thông báo",
            MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }
}

```

```

//KIỂM TRA TRÙNG MÃ KHOA
bool isDuplicate = db.Khoas.Any(k => k.MaKhoa ==
                                                NewKhoa.MaKhoa);

if (isDuplicate) {
    MessageBox.Show("Mã khoa đã tồn tại!", "Lỗi",
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}
Khoa khoa = new Khoa
{
    MaKhoa = NewKhoa.MaKhoa,
    TenKhoa = NewKhoa.TenKhoa
};
db.Khoas.Add(khoa);
db.SaveChanges();
DS_Khoa.Add(khoa);
SelectedKhoa = khoa;
MessageBox.Show("Thêm khoa thành công!", "Thông báo");
NewKhoa = new Khoa();
OnPropertyChanged(nameof(NewKhoa));
}

```

```

private void Delete()
{
    if (SelectedKhoa == null)
        return;

    if (MessageBox.Show("Bạn chắc chắn muốn xóa khoa này?",
                        "Xác nhận",
                        MessageBoxButtons.YesNo,
                        MessageBoxIcon.Question)
        != MessageBoxResult.Yes)
        return;
}

```

```

try
{
    db.Khoas.Remove(SelectedKhoa);
    DS_Khoa.Remove(SelectedKhoa);
    db.SaveChanges();
    SelectedKhoa = DS_Khoa.FirstOrDefault();
}
catch
{
    MessageBox.Show("Không thể xóa dữ liệu!!!");
}
}

```

```

private void Update()
{
    try {
        if (SelectedKhoa == null) return;
        //GÁN NGUỖC: NewKhoa → SelectedKhoa
        SelectedKhoa.MaKhoa = NewKhoa.MaKhoa;
        SelectedKhoa.TenKhoa = NewKhoa.TenKhoa;
        db.SaveChanges();
        LoadData();
        MessageBox.Show("Cập nhật thành công!");
    }
    catch (System.Exception ex) {
        MessageBox.Show("Lỗi khi cập nhật: " + ex.Message);
    }
}

```

Bước 4: Trong `QLKhoaView.xaml` (đã được tạo từ project trước)

```

<TextBlock Text="Mã khoa:"/>
<TextBox Text="{Binding NewKhoa.MaKhoa}"/>
<TextBlock Text="Tên khoa:" Margin="0 10 0 0"/>

```

```
<TextBox Text="{Binding NewKhoa.TenKhoa}"/>
<StackPanel ...>
    <Button Content="Thêm" Width="70" Margin="5"
        Command="{Binding AddCommand}"/>
    <Button Content="Sửa" Width="70" Margin="5"
        Command="{Binding UpdateCommand}"/>
    <Button Content="Xóa" Width="70" Margin="5"
        Command="{Binding DeleteCommand}"/>
</StackPanel>
```

🔊 Lưu ý: hiện tại binding dữ liệu cho textbox đang được sử dụng:

```
<TextBox Text="{Binding NewKhoa.MaKhoa}"/>
```

Nên sử dụng PropertyChanged (chuẩn hơn) như sau:

```
<TextBox Text="{Binding NewKhoa.MaKhoa, Mode=TwoWay,
    UpdateSourceTrigger=PropertyChanged}"/>
```

Trong đó:

- TwoWay (hai chiều): Dữ liệu đi **2 chiều**
 - ViewModel → UI
 - UI → ViewModel
- UpdateSourceTrigger=PropertyChanged: dùng để điều khiển khi nào dữ liệu từ View cập nhật ngược lại ViewModel (hay nói cách khác, là khi nào giá trị trong TextBox được gửi về biến trong ViewModel)

Bước 5: Thực thi kết quả.

Bài tập sinh viên tự làm

Bài 2. Ứng dụng WPF Quản lý lớp có giao diện như sau:

Mã lớp	Mã khoa
14DHKT01	KT
14DHTH01	CNTT
14DHTH02	CNTT
15DHBM01	CNTT
15DHBM02	CNTT
15DHHH01	HH
15DHHH02	HH
15DHKT01	KT
15DHKT02	KT
15DHTH01	CNTT
15DHTH02	CNTT
15DHTH03	CNTT

Yêu cầu (thực hiện tiếp tục chức năng cập nhật dữ liệu trên bài 3 – Bài thực hành 09):

- **Nút Thêm:**
 - Xóa dữ liệu trên các control nhập liệu, chuẩn bị trạng thái trống để thêm mới.
 - Bật nút Lưu và Hủy.
 - Tắt các nút Thêm, Sửa, Xóa.
- **Nút Sửa:**
 - Chọn một lớp trong DataGrid, thông tin hiện lên các TextBox.
 - Vô hiệu hóa TextBox Mã lớp (không cho chỉnh sửa), cho phép cập nhật thông tin lớp mới.
 - Bật Lưu và Hủy.
- **Nút Lưu:** lưu dữ liệu lớp mới hoặc lớp đã chỉnh sửa xuống CSDL.
- **Nút Hủy:**
 - Hủy bỏ thao tác Thêm / Sửa, khôi phục dữ liệu ban đầu.
 - Disable nút Lưu và Hủy.
 - Enable lại các nút Thêm, Sửa, Xóa.
 - Reset form về trạng thái mặc định.

- Nút Xóa: yêu cầu người dùng xác nhận và xóa lớp được chọn.
- Dữ liệu phải được đồng bộ tự động:
 - Khi chọn dòng trên DataGrid, các TextBox tương ứng sẽ hiển thị đúng dữ liệu.
 - Khi chọn Lưu/Xóa: DataGrid cập nhật lại nội dung và cập nhật xuống CSDL.
- Kiểm tra và thông báo lỗi cho người dùng khi:
 - Thêm một lớp mới mà có mã lớp trùng.
 - Sửa hoặc xóa lớp đang tồn tại sinh viên liên kết (ràng buộc khóa ngoại với bảng SinhVien).

Bài 3. Tạo ứng dụng WPF Quản lý cập nhật điểm có yêu cầu và giao diện như sau:

The screenshot shows a WPF application window titled "Quản lý nhập điểm". It features three dropdown menus at the top for "Môn học", "Năm học", and "Học kỳ". Below these are two buttons: "Tải danh sách sinh viên" and "Lưu điểm". At the bottom, there is a DataGrid with the following columns: "Mã SV", "Họ tên", "Lớp", "Điểm", and an empty column.

Yêu cầu: Sử dụng lại bài 6 trong Bài thực hành 09, bổ sung chức năng:

- **Lưu điểm**
 - Ghi điểm của từng sinh viên vào bảng KetQua: Nếu sinh viên chưa có điểm thì thêm mới, nếu sinh viên đã có điểm thì cập nhật.
 - Thông báo: “Lưu điểm thành công!” hoặc thông báo lỗi (ví dụ: điểm không hợp lệ, trống dữ liệu...).

3. Bài tập về nhà

Bài 4. Ứng dụng WPF Quản lý môn học có yêu cầu và giao diện sau:

Quản lý Môn học

Mã môn: Tên môn học:

Số TC: Tính chất:

Mã môn	Tên môn học	Số TC	Tính chất
--------	-------------	-------	-----------

Yêu cầu:

- Thực hiện đầy đủ các yêu cầu của Bài tập 2 áp dụng cho bảng MonHoc (trên bài tập 5 – Bài thực hành 9).

Bài 5. Ứng dụng WPF Quản lý sinh viên có yêu cầu và giao diện sau:

Quản lý Sinh viên

THÔNG TIN SINH VIÊN

Mã sinh viên:

Họ tên:

Giới tính: ☒ Nam ☐ Nữ

Ngày sinh:

Lớp:

DANH SÁCH SINH VIÊN

Mã SV	Họ tên	Giới tính	Ngày sinh	Mã lớp
-------	--------	-----------	-----------	--------

Yêu cầu:

- Thực hiện đầy đủ các yêu cầu của Bài tập 2 áp dụng cho bảng SinhVien (trên bài tập 4 – Bài thực hành 9)